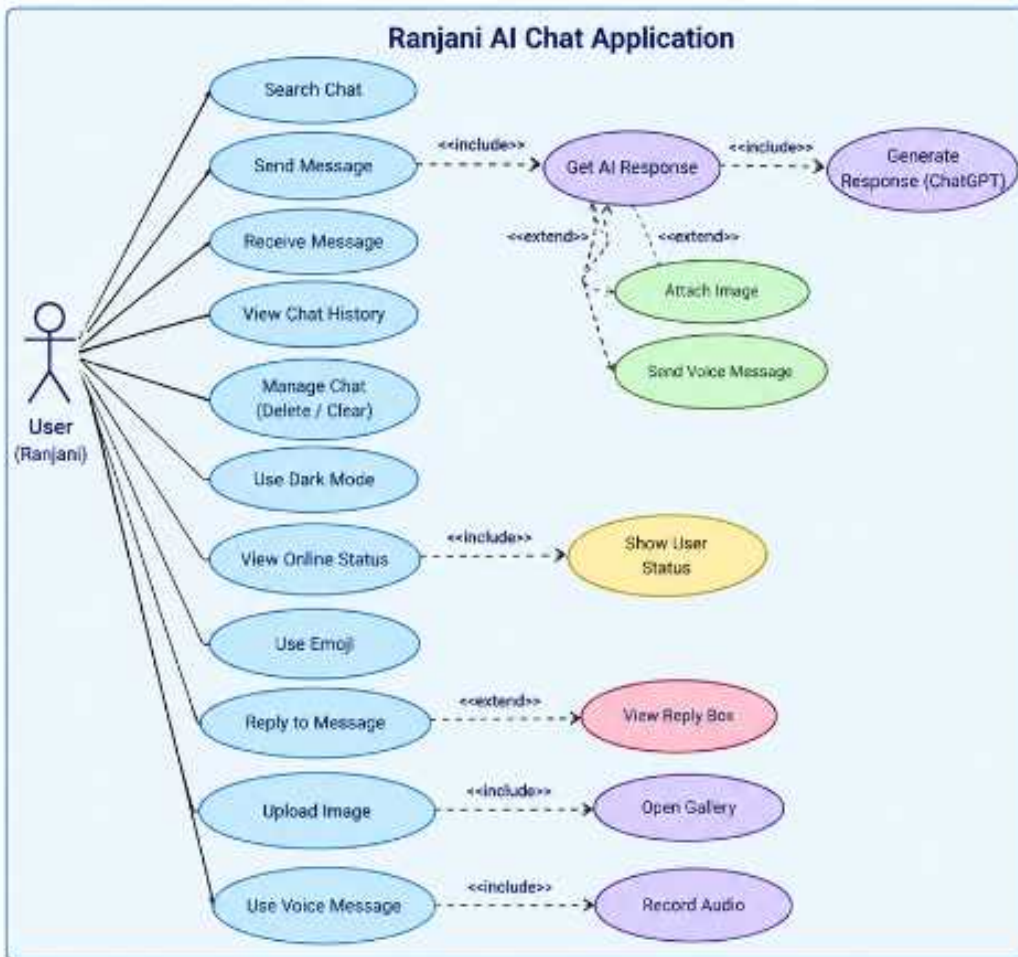




Use Case Diagram – Ranjani AI Chat Application (WhatsApp-like AI Chat App)



Use Case Descriptions

- 1 Search Chat**
User can search for a specific chat or message using the search box.
- 2 Send Message**
User types and sends a message to the AI chat. This includes text, image, voice, etc.
- 3 Receive Message**
User receives a response from the AI chat application.
- 4 View Chat History**
User can view previous chats and conversations.
- 5 Manage Chat (Delete / Clear)**
User can delete individual messages or clear the entire chat.
- 6 Use Dark Mode**
User can switch between light and dark theme.
- 7 View Online Status**
User can see if the AI is online/offline. This includes the status indicator.
- 8 Use Emoji**
User can send emojis in the chat.
- 9 Reply to Message**
User can reply to a specific message using the reply box.
- 10 Upload Image**
User can upload an image from gallery.
- 11 Use Voice Message**
User can record and send a voice message.



Short Explanation

This Use Case Diagram shows how the user interacts with the Ranjani AI Chat Application. The user can perform main actions like sending/receiving messages, viewing chat history, managing chats, and using features like dark mode, emojis, voice messages, image upload and reply. The AI response is included when the user sends a message, and it can also handle attachments like images and voice messages. The user can also check the online status of the AI.



Key Features (At a Glance)

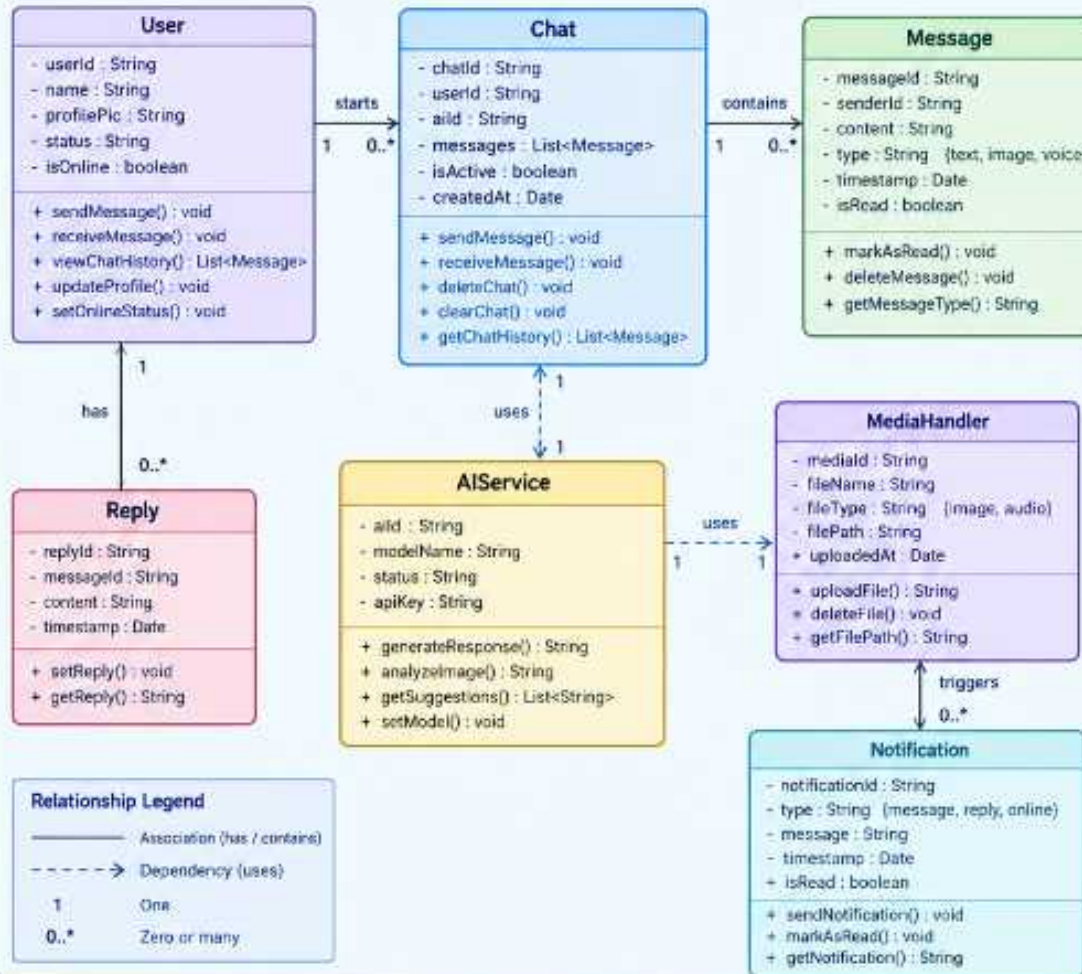
- ✓ AI powered chat (ChatGPT)
- ✓ Text, image, voice messages
- ✓ Search chat
- ✓ Chat history
- ✓ Delete / clear chat
- ✓ Dark mode
- ✓ Online status
- ✓ Emoji support
- ✓ Reply to message
- ✓ Image upload (gallery)
- ✓ Voice recording

Included / Extended Use Cases

- > <<include>> : Mandatory (always happens)
-----> <<extend>> : Optional (happens only if needed)



Class Diagram – Ranjani AI Chat Application (WhatsApp-like AI Chat App)



Class Descriptions

- 1 User**
Represents the person using the app. Stores user details and handles basic actions like sending messages, viewing chat history, etc.
- 2 Chat**
Manages a conversation between the user and AI. It contains multiple messages and maintains chat status.
- 3 Message**
Represents each message in the chat. Can be text, image, or voice message. Tracks time and read status.
- 4 Reply**
Stores the AI's reply to a user message. Linked with a specific message.
- 5 AIService**
Handles the AI functionality like generating responses, analyzing images, and providing suggestions.
- 6 MediaHandler**
Manages file uploads (images, audio, etc.) including storing and deleting media.
- 7 Notification**
Sends notifications for new messages, replies or online status updates.

Included / Extended Use Cases

Included Use Cases

- Send / Receive Message
- View Chat History
- Upload / Delete Media
- View Online Status
- Reply to Message
- Delete / Clear Chat
- Receive Notification

Extended Use Cases (Optional)

- Dark Mode
- Emoji Support
- Voice Recording
- Search Chat
- Save Chat



Short Explanation

The Class Diagram shows the main classes in the Ranjani AI Chat Application and how they are related.

- User interacts with Chat to send and receive messages.
- Chat contains many Message objects.
- Each message can have a Reply (AI response).
- AIService is used by Chat to generate responses.
- MediaHandler manages image/voice uploads and files.
- Notification is triggered when there is a new message, reply or online status.

This structure helps in organizing the data and behaviour of the application in a simple and clear way.



Activity Diagram – Ranjani AI Chat Application (WhatsApp-like AI Chat App)

User (Ranjani)

System (AI Chat App)

How it works (Step by Step)

- 1 User opens the app and enters their name (Ranjani).
- 2 Home screen (chat list) is displayed.
- 3 User types a message and taps **Send**.
- 4 System checks the message type (**text** / image / voice / emoji).
- 5 Based on the type, the system processes it and gets a response from ChatGPT.
- 6 AI response is shown in the chat box.
- 7 User can also use extra features like delete/clear chat, save chat, upload image, or record voice.
- 8 Chat continues until the user closes the app (**End**).

★ Key Features Shown in This Diagram

- Send & receive messages
- AI response using ChatGPT API
- Text, image, voice and emoji support
- Online/offline status (shown in UI)
- Delete / Clear chat
- Save chat
- Image upload
- Voice recording
- Reply to message

📁 Legend

- Start / End
- User action
- System action
- Parallel process (for different message types)
- ◇ Decision
- Flow arrow



In Short

This Activity Diagram shows the step-by-step flow of how the Ranjani AI Chat Application works. It starts from opening the app, sending a message, getting an AI response, and includes extra features like image upload, voice recording, delete/clear chat, and save chat.

Start

Open the App

Login / Enter User Name

View Home Screen (Chat List)

Type a message in the input box

Tap Send

Get user message

Check message type

Text / Image / Voice / Emoji?

Process text & get AI response

Handle image upload

Handle voice message

Handle emoji

Generate AI response (Using ChatGPT API)

Display response in chat box

Read the response

Response shown

(Optional) Use other features

Delete / Clear chat?

Save chat?

Upload image?

Record voice?

Yes

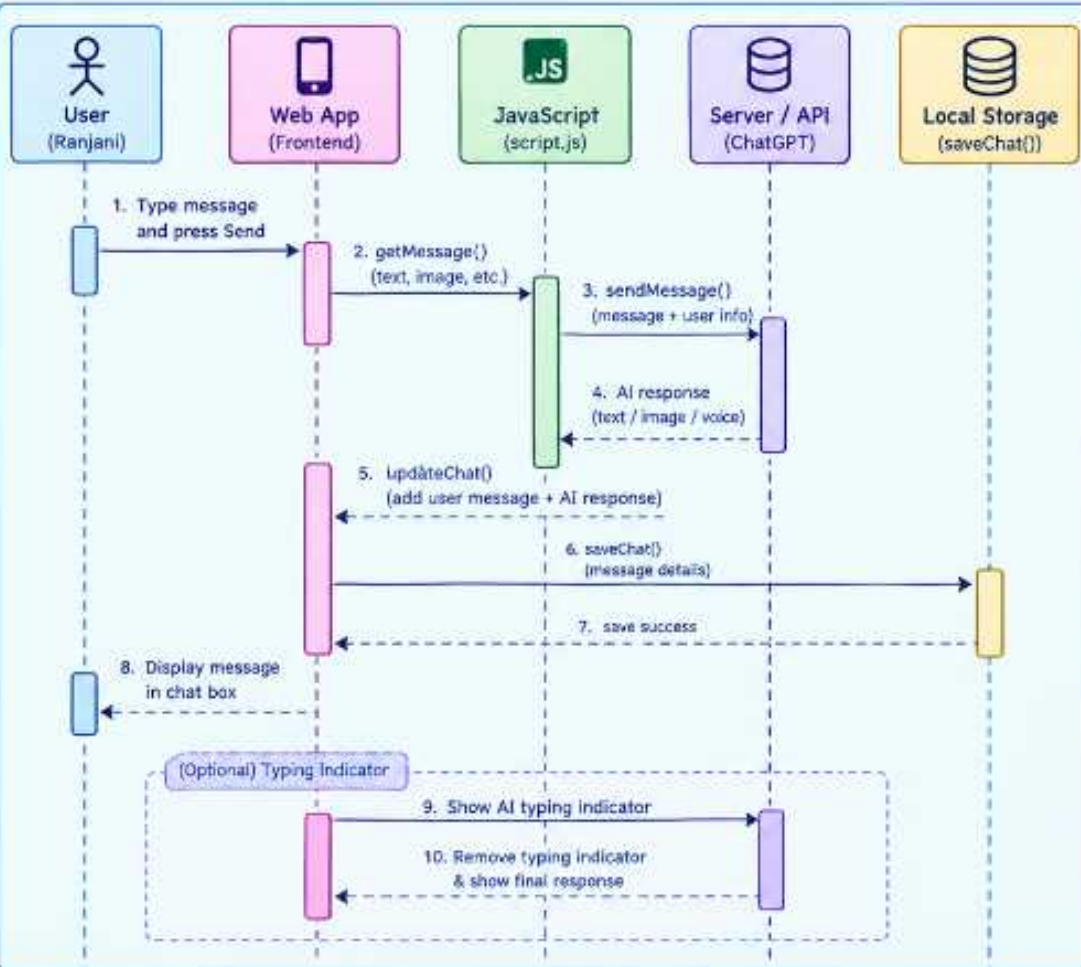
Perform the action (Delete / Clear / Save / Upload / Record)

Continue chatting

End



Sequence Diagram – Ranjani AI Chat Application (Send Message Flow)



Step-by-Step Explanation

- 1 User types a message**
Ranjani types a message in the chat box and clicks Send.
- 2 Web App gets the message**
The frontend collects the message (text, image, voice, etc.) and sends it to JavaScript.
- 3 JavaScript calls AI API**
The script.js sends the message to the ChatGPT API (server) with user details.
- 4 AI sends response**
The server returns the AI response (text / image / voice).
- 5 Web App updates chat**
The frontend updates the chat box with the user message and AI response.
- 6 Save chat (optional)**
The message details are saved using saveChat() in local storage (or backend if used).
- 7 Save success**
After saving, confirmation is received.
- 8 Message is displayed**
The final message is shown in the chat box for Ranjani.
- 9 Typing indicator (optional)**
While waiting for the AI response, "AI is typing..." is shown, and removed when the response arrives.



Key Points

- Uses client-server communication (ChatGPT API).
- Handles text, image, voice and emoji messages.
- Updates UI in real-time.
- Saves chat history using saveChat() function.
- Shows typing indicator while waiting for AI response.



Main Components Involved

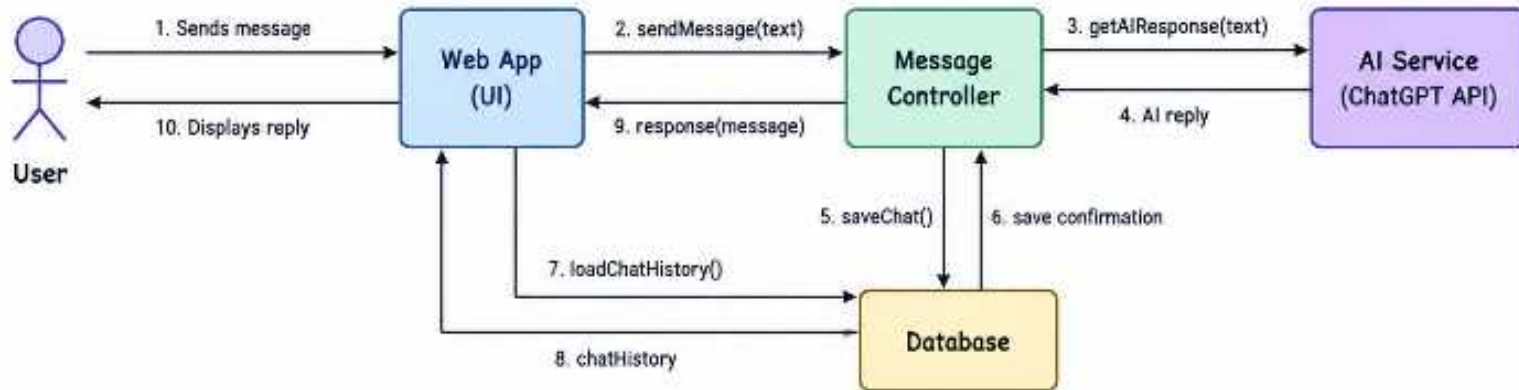
- User (Ranjani)
- Web App (HTML + CSS + JS)
- ChatGPT API (Server)
- Local Storage (or Backend)



Legend

- | | | | |
|--|------------------------|--|-----------------------|
| | User | | Method call / Request |
| | Web App (Frontend) | | Return response |
| | JavaScript (script.js) | | Lifeline |
| | Server / API (ChatGPT) | | Activation bar |
| | Local Storage | | |

Communication Diagram - AI Chat Application



Explanation

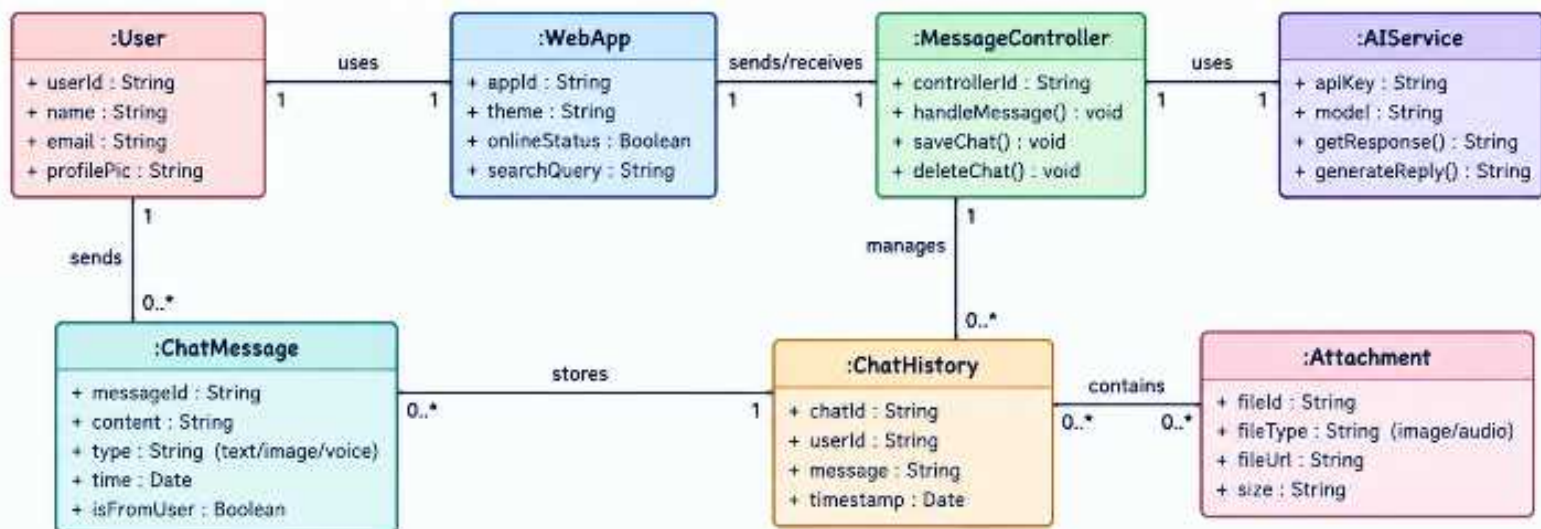
1. User sends a message through the chat box in the web app.
2. The Web App (UI) sends the message text to the Message Controller.
3. Message Controller requests an AI response from the AI Service (ChatGPT API).
4. AI Service returns the reply to the Message Controller.
5. Message Controller saves the chat (user message + AI reply) in the database.
6. Database confirms the save.
7. Web App requests the chat history from the database (when needed).
8. Database returns the chat history to the Web App.
9. Message Controller sends the final AI response back to the Web App.
10. Web App displays the reply to the user.

Key Features Shown

- User interacts with the chat through the web app (UI).
- AI Service (ChatGPT API) gives smart replies.
- Chat history is stored in a database.
- The flow shows how different components communicate with each other to complete a simple chat request.

Object Diagram – AI Chat Application

Shows the main objects (instances) in the system and how they are connected at runtime.



Explanation

1. **:User** represents the person using the chat application.
2. **:WebApp** is the front-end interface where the user interacts.
3. **:MessageController** handles the messages, saves chat, deletes chat etc.
4. **:AIService** connects to the AI model (ChatGPT API) to get responses.
5. **:ChatMessage** stores each message sent by the user or AI.
6. **:ChatHistory** maintains the conversation history for each user.
7. **:Attachment** stores any files (image, audio, etc.) shared in the chat.

Key Points

- Each object is an instance of a class in the system.
- The lines show the relationships between objects with their multiplicity (1, 0..*).
- The diagram helps to understand how different components work together at runtime.
- It is useful for visualizing the structure and flow of data in the application.